

Language Name: Indigo

Regular Expressions:

- Number Literals: `[0-9]+(\.[0-9]+)?`
- String Literals: `"(?:\"|\\|\\.)"`
- Identifiers: `[a-zA-Z_][a-zA-Z0-9_]*`

Differences from Lox:

- Changed keyword "else" to "THEN"
- Changed keyword "false" to "F"
- Changed keyword "fun" to "FUNC"
- Changed keyword "nil" to "NULL"
- Changed keyword "true" to "T"
- Changed keyword "while" to "DURING"
- Added `+=` and `-=`
- Added `#` for comments

Commands to run the scanner:

1. `javac -d bin src/indigo/*.java`
2.
 - Source File Mode: `java -cp bin indigo test/lab1/valid_tokens.indigo`
 - Interactive Mode: `java -cp bin indigo`

Setup:

```
.
└─ CIS343/
    └─ src/
        └─ indigo/
            ├── indigo.java
            ├── scanner.java
            ├── token.java
            └─ tokenType.java
        └─ test/
            └─ lab1/
                ├── errors.indigo
                └─ valid_tokens.indigo
        └─ report/
            └─ lab1.md
```

Test Cases:

- valid_tokens.indigo
 - Purpose: Test that all of the valid tokens get scanned correctly.
 - Source Input:

```
1  var my_num = 86;
2  var decimal = 8.123;
3  var my_string = "Hello World";
4  var empty_string = "";
5  var bool_t = T;
6  var bool_f = F;
7  var my_null = NULL;
8  var product = my_num * 7;
9  var quotient = decimal / 2;
10 var is_equal = (my_num == 186);
11 var is_not_equal = (decimal != 6.889);
12 var is_greater = (my_num > 100);
13 var is_less = (decimal < 10);
14 var not = !bool_t;
15 my_num += 100;
16 decimal -= 1.234;
17
18 func add_positive_numbers(a, b) {
19     if (a > 0 && b > 0) {
20         return a + b;
21     } else {
22         return NULL;
23     }
24 }
25
26 var i = 0;
27 during (i < 10) {
28     print(i);
29     i += 1;
30 }
31
32 for (var j = 0; j < 10; j += 1) {
33     print(j);
34 }
35
36 class Leah {
37     init() {
38         this.age = 20;
39     }
40 }
```

- Expected Output: Representations of the tokens without any errors
- Scanner Output:

```
VAR var null
IDENTIFIER my_num null
EQUAL = null
NUMBER 86 86.0
SEMICOLON ; null
VAR var null
IDENTIFIER decimal null
EQUAL = null
NUMBER 8.123 8.123
SEMICOLON ; null
VAR var null
IDENTIFIER my_string null
EQUAL = null
STRING "Hello World" Hello World
SEMICOLON ; null
VAR var null
IDENTIFIER empty_string null
EQUAL = null
STRING ""
SEMICOLON ; null
VAR var null
IDENTIFIER bool_t null
EQUAL = null
T T null
SEMICOLON ; null
VAR var null
IDENTIFIER bool_f null
EQUAL = null
F F null
SEMICOLON ; null
VAR var null
IDENTIFIER my_null null
EQUAL = null
NULL NULL null
SEMICOLON ; null
VAR var null
IDENTIFIER product null
EQUAL = null
IDENTIFIER my_num null
STAR * null
NUMBER 7 7.0
SEMICOLON ; null
VAR var null
IDENTIFIER quotient null
EQUAL = null
IDENTIFIER decimal null
SLASH / null
NUMBER 2 2.0
SEMICOLON ; null
VAR var null
IDENTIFIER is_equal null
EQUAL = null
LEFT_PAREN ( null
IDENTIFIER my_num null
EQUAL_EQUAL == null
NUMBER 186 186.0
RIGHT_PAREN ) null
SEMICOLON ; null
VAR var null
IDENTIFIER is_not_equal null
EQUAL = null
LEFT_PAREN ( null
IDENTIFIER decimal null
```

```
BANG_EQUAL != null
NUMBER 6.889 6.889
RIGHT_PAREN ) null
SEMICOLON ; null
VAR var null
IDENTIFIER is_greater null
EQUAL = null
LEFT_PAREN ( null
IDENTIFIER my_num null
GREATER > null
NUMBER 100 100.0
RIGHT_PAREN ) null
SEMICOLON ; null
VAR var null
IDENTIFIER is_less null
EQUAL = null
LEFT_PAREN ( null
IDENTIFIER decimal null
LESS < null
NUMBER 10 10.0
RIGHT_PAREN ) null
SEMICOLON ; null
VAR var null
IDENTIFIER not null
EQUAL = null
BANG ! null
IDENTIFIER bool_t null
SEMICOLON ; null
IDENTIFIER my_num null
PLUS_EQUAL += null
NUMBER 100 100.0
SEMICOLON ; null
IDENTIFIER decimal null
MINUS_EQUAL -= null
NUMBER 1.234 1.234
SEMICOLON ; null
FUNC func null
IDENTIFIER add_positive_numbers null
LEFT_PAREN ( null
IDENTIFIER a null
COMMA , null
IDENTIFIER b null
RIGHT_PAREN ) null
LEFT_BRACE { null
IF if null
LEFT_PAREN ( null
IDENTIFIER a null
GREATER > null
NUMBER 0 0.0
AND && null
IDENTIFIER b null
GREATER > null
NUMBER 0 0.0
RIGHT_PAREN ) null
LEFT_BRACE { null
RETURN return null
IDENTIFIER a null
PLUS + null
IDENTIFIER b null
SEMICOLON ; null
RIGHT_BRACE } null
ELSE else null
LEFT_BRACE { null
```

```
RETURN return null
NULL NULL null
SEMICOLON ; null
RIGHT_BRACE } null
RIGHT_BRACE } null
VAR var null
IDENTIFIER i null
EQUAL = null
NUMBER 0 0.0
SEMICOLON ; null
DURING during null
LEFT_PAREN ( null
IDENTIFIER i null
LESS < null
NUMBER 10 10.0
RIGHT_PAREN ) null
LEFT_BRACE { null
PRINT print null
LEFT_PAREN ( null
IDENTIFIER i null
RIGHT_PAREN ) null
SEMICOLON ; null
IDENTIFIER i null
PLUS_EQUAL += null
NUMBER 1 1.0
SEMICOLON ; null
RIGHT_BRACE } null
FOR for null
LEFT_PAREN ( null
VAR var null
IDENTIFIER j null
EQUAL = null
NUMBER 0 0.0
SEMICOLON ; null
IDENTIFIER j null
LESS < null
NUMBER 10 10.0
SEMICOLON ; null
IDENTIFIER j null
PLUS_EQUAL += null
NUMBER 1 1.0
RIGHT_PAREN ) null
LEFT_BRACE { null
PRINT print null
LEFT_PAREN ( null
IDENTIFIER j null
RIGHT_PAREN ) null
SEMICOLON ; null
RIGHT_BRACE } null
CLASS class null
IDENTIFIER Leah null
LEFT_BRACE { null
IDENTIFIER init null
LEFT_PAREN ( null
RIGHT_PAREN ) null
LEFT_BRACE { null
THIS this null
DOT . null
IDENTIFIER age null
EQUAL = null
NUMBER 20 20.0
SEMICOLON ; null
RIGHT_BRACE } null
```

```
RIGHT_BRACE } null
EOF null
```

- The scanner output matches the expected output.
- errors.indigo
 - Purpose: Tests that incorrect code gets shown as an error.
 - Source input:

```
1 var bad = @;
2 var bad2 = $50;
3 var bad3 = 100 % 2;
4 var bad3 = 90 ~ 12;
5 var bad4 = "this string is not good;
```

- Expected output: Reported errors for all 5 lines.
- Scanner output:

```
VAR var null
IDENTIFIER bad null
EQUAL = null
[line 1] Error: Unexpected character.
SEMICOLON ; null
VAR var null
IDENTIFIER bad2 null
EQUAL = null
[line 2] Error: Unexpected character.
NUMBER 50 50.0
SEMICOLON ; null
VAR var null
IDENTIFIER bad3 null
EQUAL = null
NUMBER 100 100.0
[line 3] Error: Unexpected character.
NUMBER 2 2.0
SEMICOLON ; null
VAR var null
IDENTIFIER bad3 null
EQUAL = null
NUMBER 90 90.0
[line 4] Error: Unexpected character.
NUMBER 12 12.0
SEMICOLON ; null
VAR var null
IDENTIFIER bad4 null
EQUAL = null
[line 5] Error: Unterminated string.
EOF null
```